Course Project: Fused Attention with cuTile

Optional advanced track: CuTe DSL

Project duration

4 weeks

Overview

Attention is a core operation in modern neural networks. A direct implementation computes the full score matrix, applies softmax, and then multiplies by the value matrix. This is mathematically simple but can create a large intermediate matrix and move it repeatedly between GPU memory and compute units.

In this project, you will implement the forward pass of causal scaled dot-product attention as a fused NVIDIA GPU kernel using cuTile. The kernel must compute attention in tiles and must not materialize the complete attention-score or probability matrix in global memory. You will compare your implementation with an unfused baseline and analyze how tiling, fusion, memory traffic, and numerical stability affect performance. An optional advanced track permits CuTe DSL in place of cuTile.

cuTile offers a tile-oriented programming model in which operations such as matrix multiplication and reductions are expressed over compile-time tiles. Consult the cuTile Python documentation and CUDA tile-kernel guide. Students choosing the optional CuTe DSL track should also consult the CuTe DSL overview and CuTe DSL quick-start guide.

Learning objectives

By completing this project, you should be able to:

- Decompose a compound tensor operation into GPU-sized tiles.
- Fuse matrix multiplication, scaling, masking, softmax, and value accumulation.
- Implement numerically stable online softmax.
- Reason about global-memory traffic, on-chip storage, parallelism, and synchronization.
- Design correctness tests for floating-point GPU programs.
- Benchmark GPU kernels rigorously and explain performance bottlenecks.

Problem statement

Given query, key, and value tensors Q, K, and V, compute scaled dot-product attention:

$$O = \text{softmax}\left(\frac{QK^T}{\sqrt{D}} + M\right)V$$

The tensors use the following logical layout:

- $Q \in \mathbb{R}^{B \times H \times N_q \times D}$
- $K \in \mathbb{R}^{B \times H \times N_k \times D}$
- $V \in \mathbb{R}^{B \times H \times N_k \times D_v}$
- $O \in \mathbb{R}^{B \times H \times N_q \times D_v}$

B is the batch size, H is the number of attention heads, N_q and N_k are sequence lengths, D is the query/key head dimension, and D_v is the value head dimension. For self-attention, $N_q = N_k$ and normally $D = D_v$.

The mask M is always causal. Positions with key index greater than query index are excluded by treating their score as negative infinity before softmax. This project considers causal self-attention only, so $N_q = N_k = N$.

For each query row i, your implementation must be equivalent to:

$$s_{ij} = \frac{Q_i \cdot K_j}{\sqrt{D}},$$

$$p_{ij} = \frac{e^{s_{ij}}}{\sum_t e^{s_{it}}},$$

$$O_i = \sum_j p_{ij} V_j.$$

Required scope

Your submission must:

- Implement the forward pass using cuTile, unless completing the optional CuTe DSL track.
- Use FP16 inputs and FP32 accumulation. Other input types are outside the project scope.
- Implement causal self-attention only.
- Support $D \in \{32, 64, 128\}$.
- Support sequence lengths divisible by the selected query and key tile dimensions. Arbitrary partial edge tiles are not required.
- Validate divisibility before launch and reject unsupported dimensions with a clear error.
- Fuse score computation, scaling, causal masking, softmax, and multiplication by V into one GPU kernel launch for each output tensor.
- Avoid writing the full $N_q \times N_k$ score or probability matrix to global memory.
- Use a numerically stable softmax based on running row maxima and running normalization sums.
- Include at least two tile configurations and explain how the final configuration was selected.

Dropout, backward propagation, non-causal attention, BF16, grouped-query attention, KV caching, variable-length packed sequences, and partial edge tiles are outside the required scope.

Online-softmax requirement

A stable fused kernel processes key/value tiles incrementally. For each query row, maintain:

- A running maximum m.
- A running normalization sum l.
- A running output accumulator o.

When a new score tile has row maximum m_{new}, rescale the previous state before incorporating the new tile. One valid recurrence is:

$$m' = \max(m, m_{new})$$

$$l' = e^{m - m'} l + \sum_j e^{s_j - m'}$$

$$o' = e^{m - m'} o + \sum_j e^{s_j - m'} V_j.$$

After all key/value tiles have been processed, write o/l to the output. You must explain this recurrence and its numerical-stability benefit in your report.

Input

The required test harness accepts one `.npz` file with:

- `Q`: contiguous tensor of shape `[B, H, Nq, D]` and dtype `float16`.
- `K`: contiguous tensor of shape `[B, H, Nk, D]` and dtype `float16`.
- `V`: contiguous tensor of shape `[B, H, Nk, Dv]` and dtype `float16`.

Required limits:

- $1 \le B \le 16$
- $1 \le H \le 32$
- $1 \le N \le 8192$, with N divisible by the selected query and key tile dimensions
- $D, D_v \in \{32, 64, 128\}$
- $N_q = N_k = N$ for all inputs.

The instructor may provide hidden tests using several supported, tile-divisible sequence lengths. The program must detect and clearly reject sequence lengths that are not divisible by the selected tile dimensions.

Output

Write one `.npy` file containing O in logical shape `[B, H, Nq, Dv]`. The output dtype may be FP16 or FP32, but it must be documented and used consistently.

The command-line interface must be:

```
python run_attention.py --input INPUT.npz --output OUTPUT.npy --backend cutile

# Optional advanced track
python run_attention.py --input INPUT.npz --output OUTPUT.npy --backend cutedsl
```

The process must return a nonzero exit status and a clear error message for unsupported shapes or dtypes.

Correctness requirements

Use a trusted framework implementation computed with FP32 intermediates as the reference. Test at least:

- Causal attention across every required head dimension.
- Sequence lengths equal to and larger than one tile.
- Multiple sequence lengths divisible by each tested tile configuration.
- Rejection of sequence lengths that violate the selected tile-divisibility requirements.
- Very small and relatively large score magnitudes.
- Multiple batch sizes and head counts.
- Every required head dimension.

Report maximum absolute error, maximum relative error, and a pass/fail result using a justified mixed tolerance. A recommended starting point for FP16 output is:

```
abs_error <= 2e-2 + 2e-2 * abs(reference)
```

You may select a different tolerance, but you must justify it. Tests must use fixed random seeds and must check for NaN and infinity.

Performance evaluation

Benchmark the following:

- A deliberately unfused baseline that forms the score and probability tensors.
- Your fused kernel.
- A vendor or framework attention implementation as a performance reference; it is not part of the speedup grade.

Use at least six representative tile-divisible configurations, including short and long sequences and both head dimensions 64 and 128. For every configuration:

- Run warm-up iterations before measurement.
- Time only GPU execution using CUDA events or an equivalent GPU timer.
- Synchronize correctly.
- Report the median of at least 30 measured iterations.
- Report latency, speedup over the unfused baseline, and effective output throughput.
- State the GPU model, CUDA version, DSL version, clock/power policy if known, and compilation mode.
- Exclude one-time JIT compilation from steady-state kernel timing, but report compilation time separately.

Use Nsight Compute or an equivalent profiler on at least two configurations. Discuss global-memory traffic, achieved occupancy, arithmetic throughput, memory throughput, and the most important limiting factor. Do not claim that a kernel is faster without presenting repeatable measurements.

Four-week plan

Week 1: Reference, environment, and correct fused kernel

- Set up the selected DSL and run a minimal kernel.
- Implement a framework reference and an unfused baseline.
- Build deterministic correctness tests and derive the tiled algorithm.
- Implement tiled QK^T score computation, scaling, and causal masking.
- Implement stable online softmax and accumulation with V.
- Pass small and tile-divisible shape tests, and reject unsupported shapes cleanly.

Milestone: the reference, baseline, test harness, and one correct fused configuration run end to end.

Week 2: Initial profiling and optimization

- Evaluate at least two tile configurations.
- Profile representative short- and long-sequence workloads.
- Improve memory-access patterns and on-chip data reuse.
- Reduce unnecessary conversions, synchronization, and intermediate storage.

Milestone: present an initial performance table and one profiler-guided optimization.

Week 3: Iterative profile-optimize loop

- Use profiling evidence to repeat the optimization cycle across multiple configurations.
- Tune tile dimensions, work assignment, and on-chip resource usage.
- Recheck correctness after every material optimization.
- Compare the fused kernel with the unfused baseline and performance reference.
- Record successful and unsuccessful experiments for the final report.

Milestone: demonstrate a second profiler-guided improvement and explain the remaining bottleneck.

Week 4: Robustness, final experiments, and report

- Complete the required tile-divisible shape coverage.
- Run final reproducible benchmarks and profiler captures.
- Explain performance trends, optimization history, and remaining bottlenecks.
- Clean the repository and write the final report.

Milestone: reproducible final submission with code, tests, results, and analysis.

Deliverables

Submit one repository or archive containing:

- `README.md` with setup and execution instructions.
- `run_attention.py` implementing the required command-line interface.
- Kernel source files.
- Reference and unfused-baseline implementations.
- Automated correctness tests.
- Benchmark scripts that regenerate all reported tables.
- Raw benchmark results in CSV or JSON format.
- A final report of 5–8 pages in PDF or Markdown.

The report must include the algorithm, work decomposition, tensor and tile layouts, online-softmax derivation, correctness results, benchmark method, performance plots, profiler evidence, optimization history, limitations, and individual contributions.

Possible next step

After completing the required scope, students may integrate the fused kernel into an example PyTorch model and compare it with the model's baseline attention implementation. The integration should demonstrate numerically consistent outputs and a measurable end-to-end performance improvement. This is an optional extension and is not part of the required project scope.

Optional extensions

- BF16 or FP8 input support.
- Backward propagation.
- Variable-length or packed sequences.
- Grouped-query or multi-query attention.
- Autotuning across tile configurations.

- Implementing both tracks and comparing cuTile with CuTe DSL.
- Architecture-specific pipelining or asynchronous data movement.

Optional advanced track: CuTe DSL

With instructor approval, a student or team may implement the same causal fused-attention specification using CuTe DSL instead of cuTile. This is an alternative implementation track, not an additional requirement.

The CuTe DSL track uses the same:

- Mathematical operation and causal mask.
- FP16 inputs and FP32 accumulation.
- Tile-divisibility restrictions.
- Correctness tolerances and hidden tests.
- Benchmark shapes, timing rules, and deliverables.

CuTe DSL submissions are additionally expected to explain their tensor layouts, tiled-copy and matrix-multiply abstractions, thread-level work mapping, and any architecture-specific choices. Students are not required to implement both DSLs.

Suggested references

- Original FlashAttention paper
- CuTe DSL overview
- CuTe DSL quick-start guide
- CuTe DSL programming model
- CUDA tile-kernel guide